
Disk scheduling Algo

keyword: head track
in movement

→ FCFS (First come first serve)

Disk track: 0 → 199

Initial head position: 50

Request queue:

[82, 170, 43, 140, 24, 16, 190]

sequence

50 → 82 → 170 → 43 → 140 → 24 → 16
→ 190

total head movement

50 to 82: 32

82 to 170: 88

170 to 43: 127

43 to 140: 97

140 to 24: 116

24 to 16 : 8

16 to 190 : 174

$$\therefore \text{Total head movement} = (32 + 88 + 127 + \dots + 174) = 642$$

Adv

→ Very simple

→ Fair

→ No starvation

disadv

→ high head movement

→ poor avg seek time

→ Not efficient

→ SSTF (shortest seek time first)

for head = 50

50 → 82 & 32

→ 231 ~

Individual seek distance

50 → 43 → 24 → 16 → 82 → 14
sequence

50 → 43
50 → ~~44~~: ---

AS 43 is nearest next read

is 43.

Now for head = 43

43 → 170

43 → 190

43 → 24

50 → 43 → 24 → 16 → 82 → 140 → 170 → 190
✓

→ SCAN (Elevator algorithm)

left → 0 → 199 → right

RQ: [82, 170, 43, 140, 24, 16, 190]

QA: [16, 24, 43, 82, 140, 170, 190]

→ C-SCAN

sorted RQ: [16, ~~24~~, 93, 82, 140, 170, 190]

seq

50 → 82 → 140 → 170 → 190 → 199

→ 0 → 16 → ~~24~~ → 93

movement calculation

$$50 \text{ to } 82 = 32$$

$$\vdots$$
$$190 \text{ to } 199 = 9$$

$$199 \text{ to } 0 = 199$$

$$\vdots$$
$$24 \text{ to } 93 = 19$$

$$\text{total head movement} = 391$$

→ C-Look

sorted RQ: [16, ~~24~~, 93, 82, 140, 170, 190]

50 → 82 → 140 → 170 → 190 → 16 → 24 → 93

total head movement = 34 ✓

Direct Memory Access (DMA)

1. Basic problem without DMA.

Disk → memory
→ Data packets → CPU → memory

2. Main Idea:

DMA controller

Disk → memory

I/O → memory

3. Components in DMA:

CPU

DMA controller
main memory
I/O device

4. How DMA works?

→ step 1:

CPU initializes the DMA controller.
The CPU tells the DMA controller about
→ which device will transfer data
→ starting memory address
→ amount of the data to be transferred
→ Direction of the transfer.

→ step 2:

DMA controller takes control of the BUS.

~~To transfer~~

→ step 3:

Data is transferred directly.

→ step 4:

CPU can do other work

→ step 5:

DMA sends ^{an} interrupt
when done.

Links

A link is another ~~name~~/path
that points to a file in the
file system.

soft link	Hard link
-----------	-----------

~~Shortcut~~

→ copy of the file.

~~c:\download\in.exe~~

corrupt

~~2a.txt~~

→ b.txt ✓

Differences between
soft and hard link

8 to diff

Page replacement algorithm.

→ Virtual memory

~~BSOB~~ PRA

Number of frames/buffer/slot/
total page numbers = 3

Ref. string: [7, 0, 1, 2, 0, 3, 0, 4]

→ FIFO (First in first out)

Page request	Memory frame	Page fault
7	<u>7</u>	<u>yes</u> / <u>miss</u>
0	<u>7 0</u>	yes/miss
1	7 0 1	yes/miss
2	0 1 2	yes/miss
0	0 1 2	no/hit
3	1 2 3	yes
0	2 3 0	yes
4	3 0 4	yes

Total page fault = 7

→ LRU (Least Recently Used)

PR	MF	PF
7	7	g
0	70	g
1	701	g
2	012	g
0	012	m
3	023	g
0	023	m
4	034	g

Total page fault = 6

LRTU > FIFO

→ Optimal Page Replacement :-

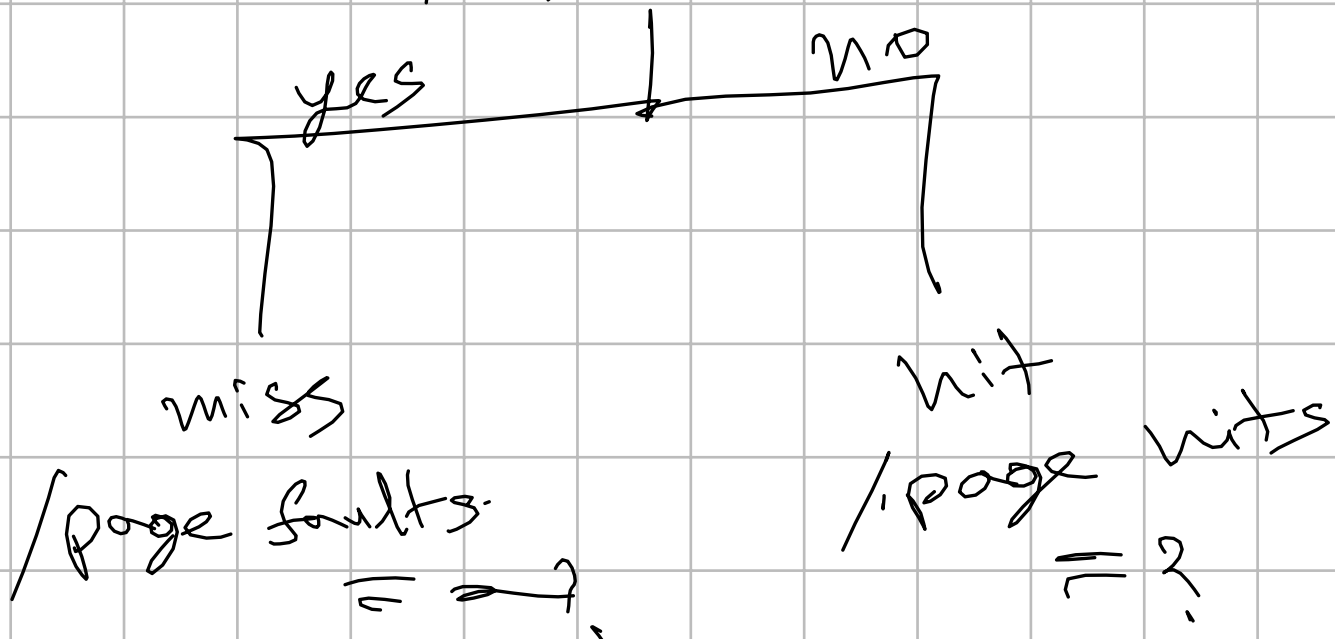
PR	MF	PF
7		8
0		8
1	<u>701</u>	8
2	0 1 2	8
0	0 1 2	n
3	0 2 3	8
0	0 2 3	n
4	0 3 4	8

Total page fault
= number of miss = 6
= hits = 2

buffer = ~~4~~ / 5

string = [7 3 4 9 2 2 7
3 2 5 7 0 4 2]
+ +

Page fault



Bankers + Analogy
next class.

Banker's Algorithm

~~1~~ $P = [\text{---}]$
 $R = [\text{---}]$

	A	B	C
P_0	1	0	1
P_1	2	1	0
P_2	1	3	2
P_3	2	2	2
P_4	0	2	1

Table: Resource Allocation

Max requirement to

	A	B	C
P ₀	3	2	2
P ₁	4	2	2
P ₂	1	4	2
P ₃	2	2	2
P ₄	1	3	1

Table: Maximum Requirement

A	B	C
3	2	2

Table: Available Resources

① Using banker's algo find if the system is in safe state and if safe then determine safe sequence.

→ Check if it's in safe state?

→ step 1: Calculate need matrix

	Maximum	Allocation	Need (Max - Alloc)
P ₀	<u>3 2 2</u>	1 0 1	<u>2 2 1</u>
P ₁	4 2 2	2 1 0	<u>2 1 2</u>
P ₂	1 4 2	1 3 2	<u>0 1 0</u>
P ₃	2 2 2	2 2 2	<u>0 0 0</u>
P ₄	1 3 1	1 2 1	<u>0 1 0</u>

For all $P[i]$ $need \leq available$
thus the system is in
safe state.

→ Step 2: Find the safe seq

we need to find where processes
satisfies $need \leq available$.

check P_0 .

$P_0[need] = 221$ and available
322

New available = $322 + 101 = \underline{\underline{423}}$

seq: $P_0 \rightarrow$

check P_1

$P_1[need] = 210 \leq available(423)$

∴ P_1 can be executed.

seq: $P_0 \rightarrow P_1$

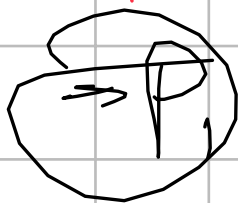
available = $423 + 210$

new available = 633

⋮
P₄

safe seq: $P_0 \rightarrow P_1 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4$

② A new request (1, 0, 1) arrives
will the system be in safe
state?



✓ $P_1[\text{request}] \leq P_1[\text{need}]$

212

✓ $P_1[\text{request}] \leq \text{available}$

322

new available = $322 - 101 = 222$

	max	Alm	need
P ₀	322	<u>101</u>	221

$$\begin{array}{c|c|c|c}
 P_1 & 22 & 210+101 & 212-101 \\
 & & = 311 & = 111 \\
 & 1 & 1 & 1 \\
 & 1 & 1 & 1 \\
 & 1 & 1 & 1 \\
 & 1 & 1 & 1
 \end{array}$$

Check $P_0 [need] \leq \text{available}$

$$221 \leq 222$$

$$\begin{array}{r}
 + 101 \\
 \hline
 = 323
 \end{array}$$

new available

$$P_0 \rightarrow P_1 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4$$

∴ As we have a safe seq. then the system is in safe state even after the new request.

Analogy / case study

A university hospital maintains a shared file system for doctors, nurses, and administrators. Recently, several problems have been reported:

- Nurses accidentally deleted shared project files. *sec*
- Doctors cannot efficiently locate departmental documents. *search*
- Some confidential test reports were accessed by unauthorized users. *security rules*
- Multiple users editing the same file caused data inconsistency.

Analyze the existing file management situation and propose appropriate file organization and protection mechanisms to solve these problems. Your answer should discuss:

- 1. Suitable directory structure
- 2. Appropriate file access methods
- 3. File sharing and concurrency control mechanisms
- 4. Security and access protection techniques
- 5. How these mechanisms improve overall system resource management

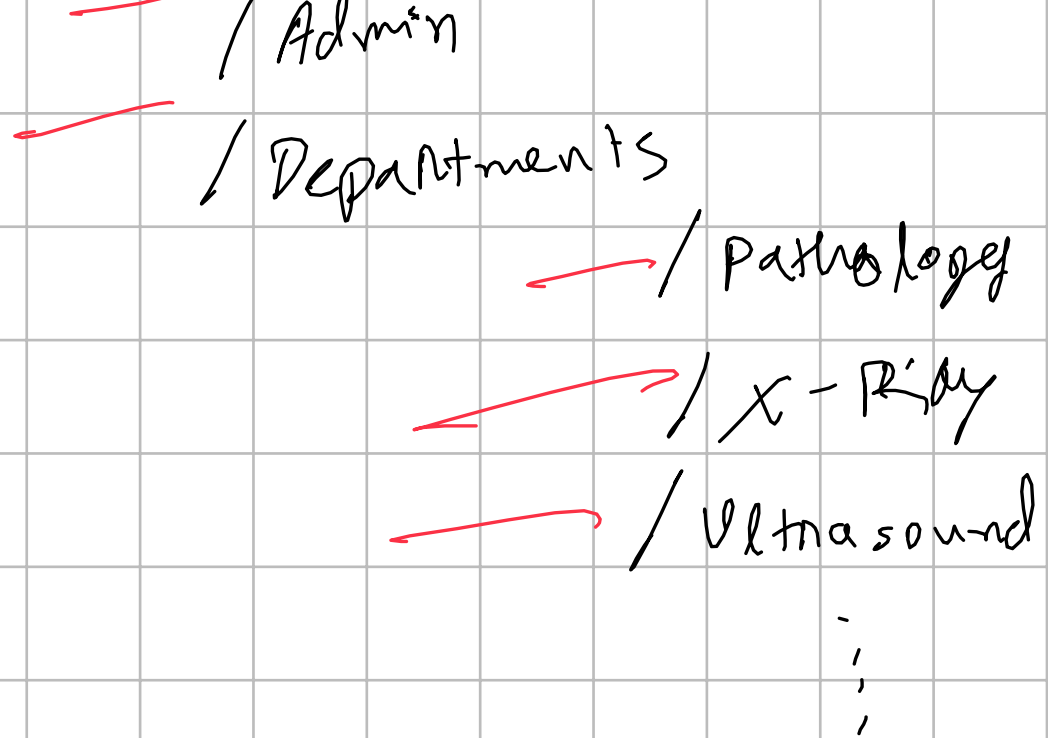
1. A hierarchical directory structure should be implemented.

Ex:

• /Hospital

 • /Doctors

 • /Nurse



Analysis: File separation 

↑ searching speed
↓ confusion

2.

